BEL TICKET LIFECYCLE MANAGEMENT AND ESCALATION MONITORING PORTAL

Complete practical guide: from a clean development machine to installation, implementation, API testing, workflow testing and deployment preparation

Version: 1.0 learning and implementation guide
Target environment: Windows development laptop; final deployment on an approved internal company LAN
Proposed stack: React + Node.js/Express + MongoDB + Socket.IO

Important: This is a practical build plan, not authorization to install software or deploy on BEL infrastructure. Obtain company IT approval first. Use only approved tools, test data, repositories, servers, ports and security settings. Do not place confidential BEL data, credentials, network details or production records into a personal account or public repository.

Prepared as a step-by-step learning document. Work through one phase at a time and verify each checkpoint before moving forward.

How to use this guide

This guide turns the requirements discussed into a practical sequence. It is intentionally staged: inspect the office laptop first, confirm company policies, install only what is missing and approved, create the project, build the backend, connect the database, build the frontend, implement role-based workflows, and test each layer.

- **Do not run every command at once.** Complete a phase, inspect its output, and then continue.
- Commands are written mainly for Windows PowerShell. If your office uses Command Prompt, most commands still work, but PowerShell-specific commands are labelled.
- The technology stack is proposed. If BEL IT mandates another database, runtime, authentication method or deployment platform, follow that standard.
- The custom ticket-number format is intentionally undecided. Use a unique database ID during early development; finalize the display format with the business owner later.
- SLA monitoring is explicitly out of Version 1.0.

Project build sequence

Phase	Outcome
0. Authorization and machine check	Approved tools, OS details, versions and constraints recorded
1. Install/check tools	Node.js, npm, Git, MongoDB tools or approved database access
2. Create project	Folders, packages, environment configuration and Git hygiene
3. Backend foundation	Express server, health endpoint, MongoDB connection
4. Data model and authentication	Users, teams, tickets, events, attachments, notifications; role checks
5. Ticket APIs	Create, edit-before-review, list, detail, review actions, escalation and fix flow
6. Audit and notifications	Every action recorded; targeted in-app notifications
7. Frontend	Login, dashboards, role-specific pages and ticket forms
8. Integration	Connect React to API; verify all roles and workflows
9. Testing	Unit, API, permission, workflow, security and offline tests
10. Deployment preparation	IT review, backup, service setup, HTTPS, LAN acceptance

1. Requirements baseline

1.1 Roles

Role	Main permissions
Testing	Create tickets; edit until DND first review; clarify returned tickets; approve/reject fixes
DND	Review; escalate; terminate; request information; return to Testing; verify fixes
Software	Work from common queue; comment; update progress; attach evidence; submit fix to DND
Admin	Manage users/roles; view all tickets; dashboards, reports and audit history

1.2 Confirmed ticket fields

Field	Rule
Ticket ID	Automatically generated; custom visible format to be decided later

Field	Rule
Ship name / ship model	Required
Category	Software or Hardware
Component name	Required
Bug title / description	Required
Severity	Critical / High / Medium / Low proposed; confirm definitions and values
Attachments	Multiple; file types and size limits set by company IT
Created by / created at	Set by authenticated server identity and server timestamp

1.3 Confirmed workflow

- Testing creates ticket. Ticket appears in DND review queue.
- Testing may edit the submitted ticket only until DND performs its first review.
- DND can escalate to Software, terminate, request more information, or return to Testing for clarification.
- Only DND can terminate. Termination reason must be mandatory.
- DND may escalate the same ticket multiple times. Every escalation is a separate event.
- Software uses a common team queue in Version 1.0; individual assignment is future scope.
- Software submits a fix to DND. DND verifies and forwards it to Testing.
- Testing can approve or reject. Approval automatically closes the ticket; rejection requires a reason.
- No SLA monitoring in Version 1.0.

1.4 Important open decisions

Open question	Why it must be answered
Exact ticket-number format	Unique ID generation, display and search
Severity definitions	Consistent prioritization
Can DND terminate at any stage or only at review?	Permission and status transition rules
After Testing rejects a fix, direct to Software or back through DND?	Core workflow routing
Can DND return to Testing more than once?	Clarification loop
Can Software submit multiple fixes?	Fix history and verification
Can resolved/terminated tickets reopen? Who may do it?	Post-closure controls
Which teams can view which tickets and attachments?	Data access and confidentiality
IT standards: OS, DB, login/SSO, HTTPS, backups, storage, allowed ports	Final architecture and deployment
Required reports, exports and concurrent user count	Dashboard and sizing

2. Office laptop and IT readiness checklist

Before installing anything, ask the company IT contact or project owner what is approved. Record the exact Windows version, whether you have administrator rights, whether software installation is restricted, and whether a company-managed Git repository is available.

- Approval to develop this internal application and use the selected stack.

- Approved development tools and software versions.
- Whether Node.js, MongoDB, Git and VS Code are already installed.
- Whether local MongoDB is allowed, or a company development database must be used.
- Whether company SSO/Active Directory integration is required.
- Approved repository and source-code handling rules.
- Approved file upload formats, maximum size, malware scanning and retention.
- Approved internal server OS, hostname, ports, HTTPS certificate and backup policy.

PowerShell — check the environment

```
# Open PowerShell. These are read-only inventory checks.
winver
$PSVersionTable
systeminfo
where.exe node
where.exe npm
where.exe git
where.exe mongod
where.exe mongosh
node -v
npm -v
git --version
mongod --version
mongosh --version
```

If a command says it cannot find a program, that tool may be absent or not in PATH. Do not immediately install it: first confirm the approved version and installation method with IT.

3. Install or verify the development tools

3.1 Visual Studio Code

Use the company-approved software portal or official installer. Confirm the approved version and extensions policy. Useful extensions may include JavaScript/TypeScript support, ESLint and Prettier, but install only if permitted.

3.2 Node.js and npm

Use the Node.js LTS version approved by IT. npm is included with Node.js. After installation, close and reopen PowerShell.

```
node -v
npm -v
where.exe node
where.exe npm
```

3.3 Git

Install the company-approved Git version if it is not present. Configure your company identity only as directed by IT.

```
git --version
git config --global user.name "Your Approved Display Name"
git config --global user.email "your-approved-company-email"
git config --list
```

Do not use a personal public GitHub repository for company source code unless the company explicitly authorizes it. Prefer the approved internal repository.

3.4 MongoDB

MongoDB is a proposed database, not a confirmed BEL standard. Ask IT whether local MongoDB Community Server is permitted or whether an approved shared development database should be used. Do not install a database server on a company device without permission.

```
# Check whether MongoDB server and shell commands are available.
mongod --version
mongosh --version

# If local MongoDB is approved and running, connect:
```

```
mongosh "mongodb://127.0.0.1:27017"
```

If the company chooses PostgreSQL, SQL Server or another database, stop here and redesign the data layer with the approved database rather than installing MongoDB anyway.

4. Create the project workspace

Use a company-approved development folder. The example below creates a folder in your user profile; change it if company policy requires another location.

```
# PowerShell
cd $HOME
mkdir BEL-Ticket-Portal
cd BEL-Ticket-Portal

mkdir docs
mkdir deployment
mkdir backend
mkdir frontend

git init
git branch -M main
```

4.1 Target folder structure

```
BEL-Ticket-Portal/
  backend/
    src/
      config/
      controllers/
      middleware/
      models/
      routes/
      services/
      utils/
    tests/
    uploads/          # local development only; do not commit
    logs/             # do not commit
    .env.example
    package.json
  frontend/
    src/
      assets/
      components/
      layouts/
      pages/
      features/
      services/
      hooks/
      styles/
    package.json
  docs/
    requirements.md
    workflow.md
    database-design.md
    api-specification.md
    test-plan.md
    deployment-guide.md
  deployment/
  .gitignore
  README.md
```

4.2 Create the .gitignore

```
# From the project root, create .gitignore
@'
node_modules/
.env
.env.*
!.env.example
uploads/
logs/
dist/
build/
coverage/
*.log
.DS_Store
```

```
Thumbs.db
'@ | Set-Content .gitignore
```

Never commit passwords, API keys, production connection strings, private certificates, real ticket attachments, logs containing sensitive information or company data.

5. Backend setup — Node.js and Express

5.1 Initialize backend

```
cd $HOME\BEL-Ticket-Portal\backend
npm init -y

npm install express cors dotenv helmet mongoose multer
npm install express-rate-limit
npm install -D nodemon
```

The package list is a starting proposal. Add authentication, validation, test and security packages only after selecting the approved authentication approach and reviewing dependencies.

5.2 Add development scripts

```
npm pkg set scripts.start="node src/server.js"
npm pkg set scripts.dev="nodemon src/server.js"
npm pkg set scripts.test="node --test"
```

5.3 Create folders

```
$dirs = @(
  "src/config", "src/controllers", "src/middleware",
  "src/models", "src/routes", "src/services", "src/utils",
  "tests", "uploads", "logs"
)
foreach ($d in $dirs) { New-Item -ItemType Directory -Force $d | Out-Null }
```

5.4 Environment configuration

```
# Create backend/.env.example (safe template; no real secrets)
@'
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://127.0.0.1:27017/bel_ticket_portal_dev
CLIENT_ORIGIN=http://localhost:5173
'@ | Set-Content .env.example

# For a local-only development environment, copy the template:
Copy-Item .env.example .env
```

Use only dummy local development values. If IT supplies a database URI or secrets, follow the approved secret-management method; do not paste them into chat or commit them.

5.5 Minimal server

```
// backend/src/server.js
require("dotenv").config();

const express = require("express");
const helmet = require("helmet");
const cors = require("cors");

const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_ORIGIN }));
app.use(express.json({ limit: "1mb" }));

app.get("/api/health", (req, res) => {
  res.status(200).json({
    status: "ok",
    service: "BEL Ticket Portal API"
  });
});

const port = process.env.PORT || 5000;
app.listen(port, () => {
  console.log(`API listening on port ${port}`);
```

```
});
```

5.6 Run and test the server

```
# From backend folder
npm run dev

# In a second PowerShell window:
curl.exe -i http://localhost:5000/api/health
```

Expected result: HTTP 200 and JSON containing status equal to ok. If the port is already in use, do not kill an unknown process; check with IT or choose an approved free development port.

6. Database connection and data design

6.1 Proposed collections

Collection	Purpose / important fields
users	username, displayName, role, teamId, active, auth reference
teams	name, code, active
tickets	ticket number, ship, category, component, title, description, severity, status, createdBy, timestamps
ticketEvents	ticketId, actor, role, action, oldStatus, newStatus, timestamp, reason, attachment refs
attachments	ticketId, originalName, storedName, mimeType, size, uploader, storage reference
notifications	recipient/team, ticketId, eventType, createdAt, readAt

6.2 Database design rules

- Use a unique database identifier for each ticket. The human-readable ticket number can be added once its format is approved.
- Use references between tickets, users, events, attachments and notifications.
- Store timestamps in a consistent UTC format; display them in the approved local timezone.
- Keep ticketEvents append-only for normal application users.
- Do not store attachment binary data in ordinary ticket documents unless the approved design specifically requires it. Store controlled file references and metadata.
- Add indexes for ticket number, status, ship, severity, createdAt and ticketId in the event collection as appropriate.
- Make ticket changes and corresponding audit events consistent; use transactions if the selected database deployment supports them.

6.3 MongoDB connection checkpoint

Do this only if local MongoDB is approved and running.

```
// backend/src/config/db.js
const mongoose = require("mongoose");

async function connectDB() {
  if (!process.env.MONGODB_URI) {
    throw new Error("MONGODB_URI is missing");
  }
  await mongoose.connect(process.env.MONGODB_URI);
  console.log("MongoDB connected");
}

module.exports = connectDB;
```

Connect before accepting requests

```
// Add near the top of server.js, after dotenv:
const connectDB = require("./config/db");
```

```
// Replace app.listen(...) with:
async function start() {
  try {
    await connectDB();
    app.listen(port, () => console.log(`API listening on ${port}`));
  } catch (error) {
    console.error("Startup failed:", error.message);
    process.exit(1);
  }
}
start();
```

```
# Run from backend
npm run dev
# Expected console: MongoDB connected, then API listening...
```

7. Authentication and role-based authorization

Do not build the production login by trusting a role sent by the browser. The backend must identify the authenticated user and load their role/team from a trusted source. Confirm whether BEL requires Active Directory/SSO, an internal identity provider or another approved login method.

7.1 Minimum authorization matrix

Action	Testing	DND	Software	Admin
Create ticket	Yes	No*	No*	Policy-defined
Edit before first DND review	Yes	No	No	Policy-defined
DND review / terminate	No	Yes	No	View/audit
Software progress / fix submit	No	No	Yes	View/audit
DND verification	No	Yes	No	View/audit
Testing approve/reject	Yes	No	No	View/audit
Manage users/roles	No	No	No	Yes

*These are proposed restrictions based on the agreed workflow. Confirm any exceptions with the project owner.

7.2 Security checklist

- Authenticate users through the company-approved identity system.
- Enforce authorization in every backend route/service.
- Reject requests when a user lacks the role or team permission.
- Validate all input on the server.
- Use secure session/token handling approved by IT; protect cookies and CSRF where applicable.
- Rate-limit login and sensitive endpoints where appropriate.
- Do not expose stack traces, secrets or database details in API responses.
- Log security-relevant events without logging passwords or tokens.

8. Ticket API plan

Build and test the API before spending time on the full UI. Example route names below are proposed and should be written into docs/api-specification.md.

Method and route	Purpose / authorization
GET /api/health	Health check; no sensitive data

Method and route	Purpose / authorization
POST /api/v1/tickets	Testing creates ticket
GET /api/v1/tickets	List tickets filtered by authorized scope
GET /api/v1/tickets/:id	Ticket details and permitted history
PATCH /api/v1/tickets/:id	Testing edits only before first DND review
POST /api/v1/tickets/:id/dnd-action	DND escalate / terminate / request info / return
POST /api/v1/tickets/:id/software-update	Software progress update
POST /api/v1/tickets/:id/submit-fix	Software submits fix to DND
POST /api/v1/tickets/:id/dnd-verify	DND forwards fix to Testing or requests correction
POST /api/v1/tickets/:id/testing-decision	Testing approves or rejects
GET /api/v1/tickets/:id/events	Authorized lifecycle history
GET /api/v1/notifications	Current user's/team's notifications
PATCH /api/v1/notifications/:id/read	Mark own notification as read

8.1 API behavior rules

- Use correct HTTP status codes: 200/201 for success, 400 for invalid input, 401 for unauthenticated, 403 for forbidden, 404 for missing, 409 for conflicting state, 500 for unexpected server failure.
- Never accept actor identity or role as trusted request-body fields. Derive them from authentication.
- Require a reason for termination and Testing rejection.
- Reject edits after DND's first review.
- Record every escalation, including repeated escalations.
- Use a controlled state-transition service; do not let a client submit arbitrary status strings.
- Paginate ticket lists; validate sort and filter fields.

9. Frontend setup — React

9.1 Create React app using Vite

```
# Open another PowerShell window
cd $HOME\BEL-Ticket-Portal\frontend
npm create vite@latest . -- --template react
npm install
npm install axios react-router-dom
npm run dev
```

If the company blocks package downloads or requires an internal package registry, stop and ask IT for the approved offline/internal package installation process.

9.2 Suggested frontend pages

- Login page using the approved authentication flow.
- Role-based dashboard.
- Create ticket form for Testing.
- Ticket list with search, filters and pagination.
- Ticket details with status, comments, attachments and history.
- DND review page with action controls.

- Software queue and ticket work page.
- DND verification page.
- Testing verification page.
- Admin users/teams, reports and audit pages.
- Notifications panel.

9.3 Suggested frontend folder layout

```
frontend/src/
  app/
    App.jsx
    routes.jsx
  components/
    Button.jsx
    StatusBadge.jsx
    FileUpload.jsx
    DataTable.jsx
  layouts/
    MainLayout.jsx
  pages/
    LoginPage.jsx
    DashboardPage.jsx
    TicketListPage.jsx
    TicketDetailsPage.jsx
    CreateTicketPage.jsx
  features/
    dnd/
    software/
    testing/
    admin/
    notifications/
  services/
    api.js
  styles/
```

The frontend may hide buttons users cannot use, but this is only a usability feature. The backend must still enforce the same permissions.

10. Ticket statuses and transition rules

Status	Meaning / next permitted actions (subject to final workflow approval)
NEW / DND_REVIEW	New ticket awaiting first DND review
RETURNED_TO_TESTING	DND requests clarification/correction
ESCALATED_TO_SOFTWARE	Ticket routed to Software queue
SOFTWARE_IN_PROGRESS	Software is investigating
FIX_SUBMITTED	Software submitted a fix to DND
DND_VERIFICATION	DND checks submitted fix
TESTING_VERIFICATION	DND sent fix to Testing
REOPENED	A rejected fix or authorized reopen needs more work
RESOLVED	Testing approved; automatically closed
TERMINATED	DND terminated with reason

The precise rejection route and whether DND may terminate at later stages remain open. Do not hard-code those assumptions until the owner approves them.

11. Audit history implementation requirements

Every business action should be handled by a service that validates the current status, validates the user's role, updates the ticket and creates a TicketEvent. Keep event creation separate from editable ticket fields.

```
# Example event shape (illustrative JSON, not a ready-to-run command)
{
  "ticketId": "database-ticket-id",
  "actorUserId": "authenticated-user-id",
  "actorRole": "DND",
  "action": "ESCALATED_TO_SOFTWARE",
  "oldStatus": "DND_REVIEW",
  "newStatus": "ESCALATED_TO_SOFTWARE",
  "reason": "Investigation required",
  "createdAt": "server-generated timestamp"
}
```

- Use server-generated timestamps.
- Record actor identity from the authenticated session.
- Do not allow normal users to update/delete events.
- Keep repeated escalations as separate event documents.
- Test that failed ticket updates do not leave misleading audit events.
- Restrict direct database access and use separate least-privilege service credentials.

12. Notifications and real-time updates

Start with persistent in-app notifications. After that works, add Socket.IO or another company-approved real-time mechanism so the UI updates without refresh.

Event	Recipient
New ticket	DND
DND escalation	Software
Termination / clarification / return	Testing
Software progress / fix submitted	DND
DND forwards to Testing	Testing
Testing rejection	DND and Software
Testing approval	DND and relevant participants
Important events	Admin

Do not send every notification to every user. Store recipient, ticket reference, event type, created time and read time. Notifications are not a substitute for audit history.

13. Attachment handling

- Ask IT for allowed extensions, maximum file size, per-ticket count, total storage quota and retention rules.
- Use a server-generated stored filename; preserve the original name only as metadata.
- Validate file extension, MIME type and file signature where possible.
- Store outside the public web root and stream downloads only after authorization.
- Apply malware scanning if required by IT.
- Prevent path traversal and filename injection.
- Record upload and download-relevant audit events according to policy.
- Do not use real BEL documents for development testing unless explicitly approved.

14. Testing plan — do not skip this

14.1 Test environment

Use a separate development database and fabricated test users/tickets. Never run destructive tests against production. Take a backup before any test that modifies shared data.

14.2 Health and API smoke tests

```
# Backend running in one terminal:
npm run dev

# Second terminal:
curl.exe -i http://localhost:5000/api/health
```

14.3 Authentication and permission tests

Test	Expected result
No login calls protected ticket API	401 Unauthorized
Testing user calls DND terminate endpoint	403 Forbidden
Software user calls Testing approve endpoint	403 Forbidden
DND user terminates with no reason	400 validation error
Testing edits ticket before DND review	Success
Testing edits ticket after first DND review	409 or 403; no change
User requests ticket outside permitted scope	403 or 404 according to security policy

14.4 Workflow acceptance tests

Test scenario	Expected result
Testing creates valid ticket	Ticket saved; unique ID; DND notified; audit event created
Testing omits required title	Validation error; no ticket created
DND escalates ticket	Software queue updated; escalation event and notification
DND escalates same ticket again	Second distinct escalation event exists
DND terminates ticket	TERMINATED status; mandatory reason in history
Software updates progress	Progress saved; DND notified; event recorded
Software submits fix	DND verification queue; event recorded
DND forwards fix	Testing verification queue; event recorded
Testing rejects without reason	Validation error; no status change
Testing rejects with reason	Rejection recorded; follows approved return route
Testing approves	Status becomes RESOLVED automatically; closure event recorded

14.5 Attachment tests

- Upload an allowed file under the approved size limit — should succeed.
- Upload a file larger than the approved limit — should be rejected safely.
- Upload a disallowed file type — should be rejected.
- Attempt to download an attachment without permission — should be denied.

- Check that stored filenames cannot overwrite another file.
- Check that deleting a ticket does not accidentally delete required audit evidence; deletion/retention policy must be approved.

14.6 Offline/LAN tests

- Deploy to an approved test server in a disconnected or controlled LAN test environment.
- Disconnect internet access and verify login, ticket creation, search, attachment access and notifications still work.
- Check browser developer tools for external CDN, font, API or telemetry requests; there should be no required external dependency.
- Test access from another approved LAN workstation.
- Verify only approved ports are listening and firewall rules are documented.

14.7 Performance and recovery tests

- Test with a representative number of dummy tickets and attachments.
- Test pagination and common filters.
- Check application logs for errors and sensitive data leakage.
- Verify backup and restore using a non-production test database.
- Test restart behavior and service recovery with IT.

15. Useful development and troubleshooting commands

15.1 Common commands

```
# Show current folder
Get-Location

# List files
Get-ChildItem

# Enter project
cd $HOME\BEL-Ticket-Portal

# Backend dependencies
cd backend
npm install

# Run backend
npm run dev

# Frontend dependencies
cd ..\frontend
npm install

# Run frontend
npm run dev

# Check processes listening on a port
Get-NetTCPConnection -LocalPort 5000 -State Listen

# Check HTTP endpoint
curl.exe -i http://localhost:5000/api/health
```

15.2 Git commands

```
# From project root
git status
git add .
git commit -m "Initial approved project structure"

# Inspect recent history
git log --oneline -5

# Before pushing, confirm the remote is the company-approved repository:
```

```
git remote -v

# Only after approval and repository setup:
git push -u origin main
```

15.3 Common problems

Problem	Checks
node/npm not recognized	Close/reopen terminal; check installation and PATH; contact IT
Port already in use	Check Get-NetTCPConnection; use approved alternate port
MongoDB connection refused	Check whether local DB is approved, service is running and URI/port are correct
MONGODB_URI missing	Confirm backend/.env exists and dotenv loads before connection
CORS error	Check exact frontend origin and approved CORS configuration
401 / 403	Check login/session and backend role/permission mapping
404 route	Check HTTP method, route path and router mounting
File upload fails	Check multipart field name, upload limit, directory permissions and validation

16. Deployment preparation — IT-led

Do not deploy to a BEL production server yourself unless explicitly authorized. Development on a laptop and production hosting are separate activities.

- Confirm server OS, CPU/RAM/storage, virtualization and approved runtime.
- Confirm database hosting, backup, restore, replication and retention.
- Confirm company identity provider and account lifecycle.
- Confirm HTTPS certificate, DNS/internal hostname, firewall ports and reverse proxy.
- Confirm service account, least privilege, log rotation and monitoring.
- Confirm attachment storage capacity, malware scanning, retention and backup.
- Prepare an installation and rollback plan.
- Perform security review, vulnerability/dependency review and user acceptance testing.
- Document support owner, escalation contact and maintenance process.

Example local build commands

```
# Build React frontend
cd $HOME\BEL-Ticket-Portal\frontend
npm run build

# Backend production dependencies (after implementation is complete)
cd ..\backend
npm ci --omit=dev

# These commands prepare artifacts only. They do not deploy the system.
```

Do not expose a development server directly to the company LAN. Production hosting, HTTPS termination, reverse proxy, service management and firewall configuration must be designed and approved by IT.

17. Suggested day-by-day office execution plan

Session	Tasks and evidence to save
1	IT approval; record Windows and tool versions; check Node/npm/Git/database availability

Session	Tasks and evidence to save
2	Create project folders, README, .gitignore and requirements/workflow documents
3	Initialize Express; health endpoint; test with curl
4	Connect approved development database; verify connection and failure handling
5	Design User, Team, Ticket and TicketEvent schemas; review with project owner
6	Implement authentication integration and backend role checks
7	Implement ticket creation, validation and list/detail APIs
8	Implement pre-DND edit rule and DND actions
9	Implement Software queue, progress updates and fix submission
10	Implement DND verification and Testing approve/reject workflow
11	Implement audit consistency, notifications and attachment controls
12	Build React login, dashboards, ticket forms and detail pages
13	Complete role-specific pages and API integration
14	Run API, permission, workflow, attachment and offline tests; fix defects
15	Prepare IT deployment package, documentation and acceptance demo

18. Documentation to maintain in the repository

- README.md — purpose, prerequisites, approved setup, start commands.
- docs/requirements.md — confirmed requirements and open decisions.
- docs/workflow.md — workflow diagram and status transition table.
- docs/database-design.md — collections, fields, indexes and relationships.
- docs/api-specification.md — endpoint, request, response, permission and error details.
- docs/test-plan.md — test cases, actual results, defects and retest evidence.
- docs/deployment-guide.md — IT-approved deployment and rollback instructions.
- docs/decision-log.md — decision, owner, date, rationale and status.

Daily progress template

```
Date:
Phase / task:
What I intended to do:
Commands I ran:
Files I created or changed:
What worked:
Exact error (if any):
What I learned:
Test performed and result:
Next step:
```

19. Safe prompts for using ChatGPT during development

Use these prompts with an approved AI tool only. Do not paste confidential source code, real ticket records, credentials, internal IPs, network diagrams or company documents unless company policy explicitly permits it.

```
PROMPT 1 — Continue from my current checkpoint
I am building an internal ticket lifecycle portal. Explain the next
single step first. Tell me which terminal/folder to use, the exact
command, what it does, expected output, and how to verify it.
```

```
Wait for my output before continuing.

PROMPT 2 — Explain code before writing it
I am learning Node.js/Express. Explain the purpose of each file,
folder, import, function, route, middleware and database operation.
Then provide the smallest working implementation and a test.

PROMPT 3 — Debug
Here is a sanitized error and the command that produced it.
Explain the likely cause, ask for any missing non-sensitive detail,
and give one safe diagnostic step at a time. Do not guess.

PROMPT 4 — API test
For this endpoint, provide success, validation, unauthorized,
forbidden, not-found and invalid-state tests. Use dummy data only.

PROMPT 5 — End-of-day record
Summarize today's completed work, commands, files changed,
tests passed/failed, unresolved issues and the exact next step.
```

20. Final readiness checklist

- Company project approval and approved toolchain confirmed.
- Development laptop OS and installed versions recorded.
- Approved source-code repository selected.
- All open workflow questions assigned to an owner.
- Database and authentication standards approved.
- Ticket status transitions and role permissions reviewed.
- Audit event and notification behavior tested.
- Attachment policy supplied by IT.
- API and UI tests pass using dummy data.
- Offline/LAN test completed in an approved environment.
- Backup, restore, deployment and rollback reviewed by IT.
- User acceptance test signed off by business owners.

Appendix A — first-day command checklist

```
# 1. Open PowerShell and inspect
winver
$PSVersionTable
node -v
npm -v
git --version
mongod --version
mongosh --version

# 2. Create project only after approval
cd $HOME
mkdir BEL-Ticket-Portal
cd BEL-Ticket-Portal
mkdir docs, deployment, backend, frontend
git init
git branch -M main

# 3. Backend initialization
cd backend
npm init -y
npm install express cors dotenv helmet mongoose multer
npm install -D nodemon
npm pkg set scripts.start="node src/server.js"
npm pkg set scripts.dev="nodemon src/server.js"

# 4. Frontend initialization
cd ..\frontend
npm create vite@latest . -- --template react
```

```
npm install
npm install axios react-router-dom

# 5. Run separately in two terminals
# Terminal 1:
cd $HOME\BEL-Ticket-Portal\backend
npm run dev

# Terminal 2:
cd $HOME\BEL-Ticket-Portal\frontend
npm run dev
```

The first-day checklist assumes the proposed stack has been approved and the packages can be installed from an approved source. Do not run the installation steps until those conditions are met.

Appendix B — expected development URLs

Service	Typical local development URL
React/Vite frontend	http://localhost:5173
Express API	http://localhost:5000
Health check	http://localhost:5000/api/health
MongoDB local service	mongodb://127.0.0.1:27017

These are local development examples only. Production URLs, ports, hostnames and database addresses must come from company IT.